

Generative Quantum Machine Learning

Rishi Vora
IISER Mohali

28 April 2026

SUPERVISOR:
M. S. Santhanam
IISER Pune

Abstract

Generative models have recently emerged as a powerful tool for learning and sampling from complex quantum state distributions. In this work, several architectures are investigated and compared, including Quantum Direct Transport (QuDT) and Quantum Denoising Diffusion Probabilistic Models (QuDDPM). A hybrid variant, Sequential Quantum Direct Transport (S-QuDT), is introduced, demonstrating significant improvement in efficiency when learning quantum states. To ensure efficient training, a comprehensive benchmarking of gradient-free and gradient-based optimizers is performed. Performance optimizations and a simulation framework developed in **Julia** are detailed to scale towards 10-qubit systems, with a focus on learning the localised states of the Quantum Kicked Rotor system, a pragmatic model in quantum chaos.

1 Introduction

The preparation and characterization of complex quantum states are fundamental challenges in quantum information science. While quantum computers promise to simulate many-body systems and solve optimization problems that are classically intractable, the practical utility of these devices often depends on the ability to prepare specific, high-fidelity quantum states, such as ground states of Hamiltonians or thermal states. Generative Quantum Machine Learning models have shown promise in learning and sampling from complex quantum state distributions.

1.1 Problem Statement

Consider a target distribution $\mathcal{E}_0$ of pure quantum states in a Hilbert space $\mathcal{H}$. The task is to generate new states from this distribution, given only a finite number of samples $\mathcal{S}_0 = \left\{ \left| \psi_i^{(0)} \right\rangle \right\}_{i=1}^m$ from that distribution.

The training process involves finding the optimal parameters θ^* that minimize a distance measure $\mathcal{L}$ between the generated and target distributions:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\mathcal{E}_{\text{gen}(\theta)}, \mathcal{E}_0) \quad (1)$$

Achieving this requires not only a sufficiently expressive circuit ansatz but also a distance metric that can faithfully capture the geometric structure of the quantum state space and an efficient optimization strategy to navigate the resulting loss landscape.

2 Distance Metrics

Given the nature of the problem, fidelity-based distance metrics are used. Two such loss functions and their properties are described [1]. Let $\mathcal{E}_1$ and $\mathcal{E}_2$ be two state distributions, and $\mathcal{S}_1$ and $\mathcal{S}_2$ be finite sets of size n_1 and n_2 sampled from these distributions respectively:

2.1 Maximum Mean Discrepancy (MMD)

$$\mathcal{D}_{\text{MMD}}(\mathcal{S}_1, \mathcal{S}_2) = \bar{F}(\mathcal{S}_1, \mathcal{S}_1) + \bar{F}(\mathcal{S}_2, \mathcal{S}_2) - 2\bar{F}(\mathcal{S}_1, \mathcal{S}_2) \quad (2)$$

where

$$\bar{F}(\mathcal{S}_1, \mathcal{S}_2) = \mathbb{E}_{|\varphi\rangle \in \mathcal{S}_1, |\psi\rangle \in \mathcal{S}_2} |\langle \varphi | \psi \rangle|^2 \quad (3)$$

is the mean fidelity. MMD is easy to compute and differentiable, but compares feature averages and hence may not capture the full structure of the distribution.

2.2 2-Wasserstein Distance (W_2)

Assuming that the states are uniformly sampled, the Wasserstein distance of second order amounts to solving the following discrete optimal transport (OT) problem:

$$\begin{aligned} W_2(\mathcal{S}_1, \mathcal{S}_2) &= \min_{\Gamma} \langle \Gamma, C \rangle, \\ \text{s.t. } \Gamma \mathbf{1}_{n_1} &= \frac{1}{n_1} \mathbf{1}_{n_1}, \\ \Gamma \mathbf{1}_{n_2} &= \frac{1}{n_2} \mathbf{1}_{n_2}, \\ \Gamma_{ij} &\geq 0 \end{aligned} \quad (4)$$

where

$$C_{ij} := 1 - |\langle \varphi_i | \psi_j \rangle|^2 \quad (5)$$

and Γ is a transport plan, C is the cost matrix, $\langle \cdot, \cdot \rangle$ is the Frobenius inner product, and $|\varphi_i\rangle \in \mathcal{S}_1, |\psi_j\rangle \in \mathcal{S}_2$. Wasserstein distance captures the geometry of the underlying space and is a more faithful measure of distributional similarity. However, finding the optimal transport plan Γ^* is a linear programming problem with $\mathcal{O}(n^3)$ complexity.

While an alternative called Sinkhorn distance [2] has reduced complexity by introducing an entropic regularization term, it has several downsides:

- Careful tuning of the regularization parameter is required.
- Specially for generative model learning, it causes shrinkage of the learned distribution towards the mean, and therefore cannot adequately cover the whole support of the target distribution.

2.3 IPOT framework

To compute the Wasserstein distance more efficiently, the Inexact Proximal point method for Optimal Transport (IPOT) [3] is used. It is an iterative algorithm with theoretical convergence guarantees.

Theoretically, it has similar complexity as the Sinkhorn method. Empirically, the algorithm seems to be linearly convergent with just one inner iteration, and thus is well suited for the present use case.

For use in a gradient-based optimization approach, the gradient of the Wasserstein distance with respect to the parameters of the generative model is required. This can be obtained in two ways:

- **Automatic differentiation:** The gradient can be computed by backpropagating through the proximal point iterations in IPOT. However, this is computationally expensive and memory intensive, especially for large sample sizes.
- **Envelope theorem:** Backpropagating through the proximal point iterations can be skipped by using the envelope theorem. The theorem statement and the following derivation are adapted for the present use case from Appendix B in [3]:

Theorem 2.1 (Envelope theorem). *Let $f(x, \theta)$ and $l(x)$ be real-valued continuously differentiable functions, where $x \in \mathbb{R}^n$ are choice variables and $\theta \in \mathbb{R}^m$ are parameters. Denote x^* to be the optimal solution of f with constraint $l = 0$ and fixed θ , i.e.*

$$x^* = \underset{x}{\operatorname{argmin}} f(x, \theta) \quad \text{s.t.} \quad l(x) = 0. \quad (6)$$

Then, assume that V is continuously differentiable function defined as $V(\theta) \equiv f(x^(\theta), \theta)$, the derivative of V over parameters is*

$$\frac{\partial V}{\partial \theta} = \frac{\partial f}{\partial \theta} \quad (7)$$

In the present case:

- $V(\theta)$ is the Wasserstein distance W_2 .
- The choice variable x is the transport plan Γ .
- The objective function is $f(\Gamma, \theta) = \langle \Gamma, C \rangle = \sum_{ij} C_{ij} \Gamma_{ij}$.

According to the theorem,

$$\frac{\partial W}{\partial \theta} = \sum_{ij} \Gamma_{ij}^* \frac{\partial C_{ij}(\theta)}{\partial \theta} \quad (8)$$

For the cost specified as per Equation 5,

$$\frac{\partial C_{ij}}{\partial \theta} = \frac{\partial}{\partial \theta} \left(1 - |\langle \varphi_i | \psi_j(\theta) \rangle|^2 \right) \quad (9)$$

Substituting this back into the gradient expression, the following is obtained:

$$\frac{\partial W}{\partial \theta} = - \sum_{ij} \Gamma_{ij}^* \frac{\partial}{\partial \theta} |\langle \varphi_i | \psi_j(\theta) \rangle|^2. \quad (10)$$

So the gradient of the Wasserstein distance can be computed by just backpropagating through the fidelity term, without backpropagating through the proximal point iterations in IPOT, which is much more efficient.

It is also worth noting that calculating the fidelity of two pure states can be performed directly on a quantum computer using the SWAP test as described in [1], making it a promising distance metric for training generative models on quantum hardware in the future.

3 Generative Model Architectures

The problem and the distance metrics having been defined, the existing generative models are described, followed by the introduction of the proposed hybrid model.

3.1 Quantum Direct Transport (QuDT)

The most trivial approach can be manifested in the form of the Quantum Direct Transport (QuDT) model [1], which learns a direct mapping from the initial distribution of quantum states to the target distribution. This is achieved by training a single, deep Parameterized Quantum Circuit (PQC) to directly transport the initial states to the target states in one step.

Let a toy problem be specified: learning to generate the distribution of 1-qubit states clustered around any arbitrary state, starting from a Haar random ensemble.

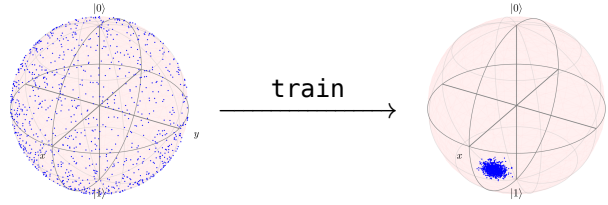


Figure 1: The overarching goal of the toy problem.

A simple PQC is trained to learn to generate this distribution. It consists of L layers where each layer consists of single qubit rotations using learnable R_X and R_Y gates, followed by entangling control- Z gates on nearest neighbours. Additionally, n_a ancillary qubits are included, initialized in the state $|0\rangle^{\otimes n_a}$ and measured out at the end of the circuit. The ancillary qubits increase the expressivity of the model, but the reasoning behind their use will be clear in Section 3.2.

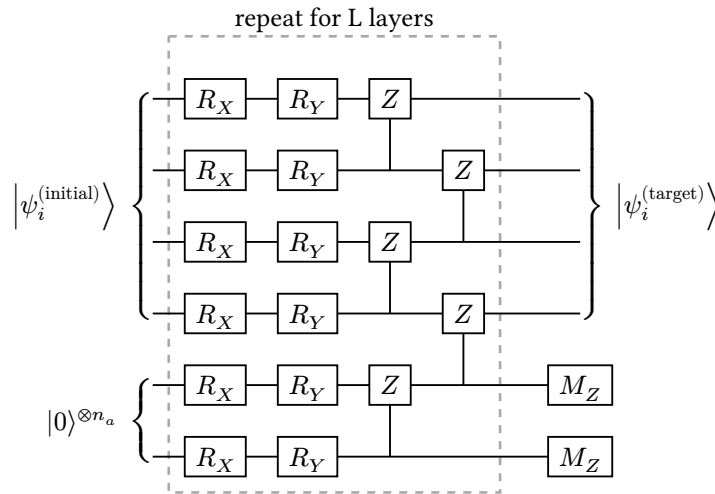


Figure 2: The QuDT circuit. M_Z represents measurement in the Z basis.

3.2 Quantum DDPM

Using the same core concept as classical Diffusion models (Appendix A), a quantum version of DDPM called Quantum Denoising Diffusion Probabilistic Models (QuDDPM) was proposed [1]. An ensemble of states is scrambled over multiple steps in the forward process and then a series of PQCs are trained to reverse this scrambling process step by step in the reverse process. This divides the task of learning a map from full noise to the target distribution into subtasks with low-depth circuits which can avoid barren plateaus.

3.2.1 The forward process

This consists of T steps, where at each step t a random unitary $U_i^{(t)}$ is applied to each state i in the ensemble, which scrambles them. After T steps, the states are completely scrambled and form a Haar random ensemble.

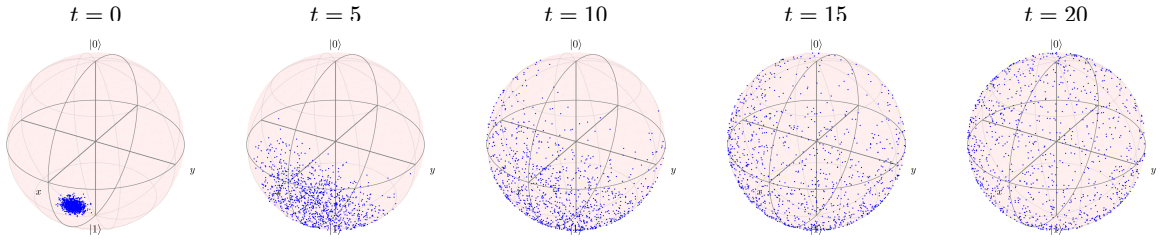
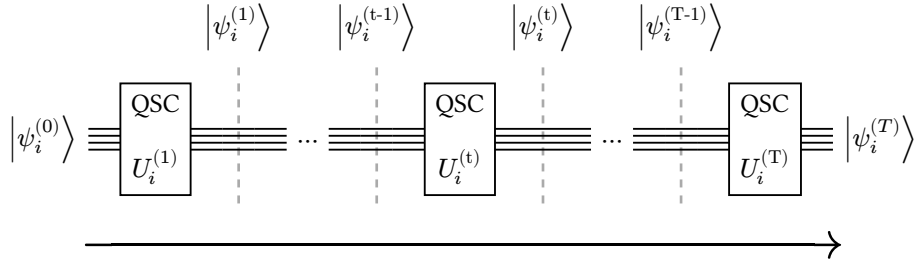
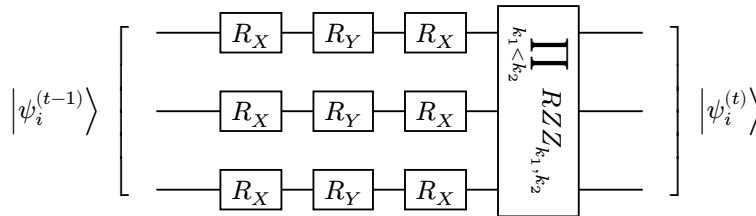


Figure 3: Scrambling $m = 1000$ states sampled from the toy distribution over $T = 20$ steps, as shown on the Bloch sphere.

In practice, U_t is implemented using the fast scrambling model [4] — layers of tunable single-qubit rotation gates followed by homogeneous entangling layers of all-to-all ZZ rotations — and the hardware efficient ansatz [5], which is as follows:



(a) Forward process circuit



(b) Quantum Scrambling Circuit (QSC) Block

Figure 4: The forward circuit (Figure 4a) consists of T number of QSC blocks. Each QSC block (Figure 4b) consists of extrinsic single qubit rotations using Euler angles followed by homogeneous entangling layers of all-to-all ZZ rotations.

3.2.2 The reverse process

The reverse process consists of T steps, where at each step t a PQC is trained to learn the denoising map from the scrambled distribution at step t to the less scrambled distribution at step $t - 1$. The reverse circuit is as follows:

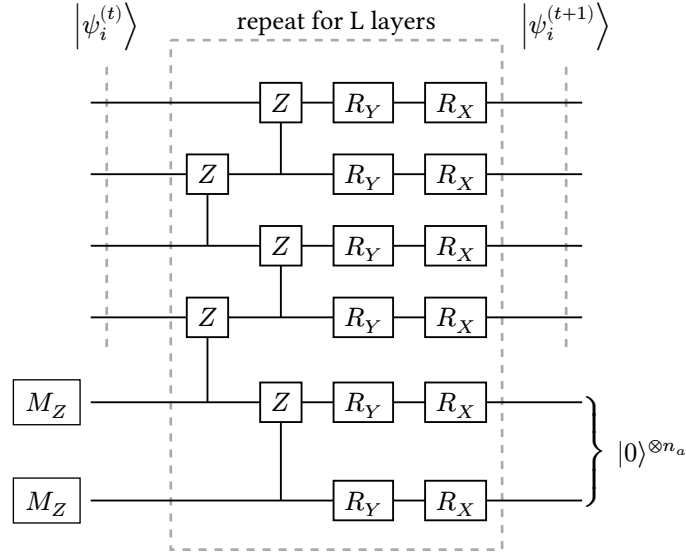
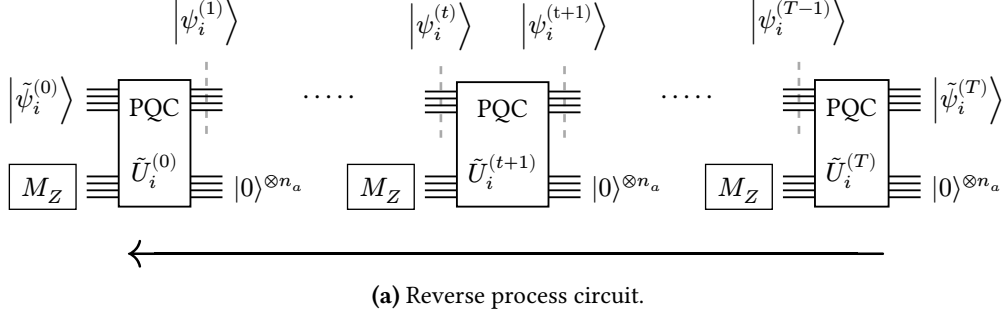


Figure 5: The reverse circuit (Figure 5a) consists of T number of PQC blocks. Each PQC block (Figure 5b) is the same as the one used in the QuDT model, but with its own learnable parameters.

Measurements are necessary, as the denoising map is contractive and maintains the purity of all generated data in $\tilde{S}_0$. No specific constraint is placed on the measurement result. The measurement on ancillas is performed and discarded; only the post-measurement state of the data qubits is collected [1].



Connection with QuDT

The QuDT model can be seen as a special case of QuDDPM with $T = 1$ step in the reverse process, where the forward process is just a single random unitary that scrambles the initial distribution. This justifies the choice of ancillary qubits and their measurement in the QuDT model.

3.3 Sequential Quantum Direct Transport (S-QuDT)

A hybrid model, Sequential Quantum Direct Transport (S-QuDT), is proposed, taking ideas from both QuDT and QuDDPM. The model consists of T steps, where at each step t a PQC is trained to learn a direct transport map from the distribution at step t to the target distribution, instead of learning a denoising map to the less scrambled distribution as in QuDDPM.

This removes the forward scrambling process entirely. Empirically, it is found that this model can achieve similar convergence performance as QuDDPM, while requiring significantly fewer steps and thus much less training time. The reverse circuit used in S-QuDT is the same as the one used in the QuDDPM model (Figure 5a and Figure 5b). Ancilla qubits are still used and measured out at the end of each step, as in QuDDPM, for the same reasons.

4 Optimizers

To find the optimal parameters for the generative model, an optimizer is employed. These are classified into two categories: gradient-free and gradient-based optimizers.

4.1 Gradient-based optimizers

The core idea is to traverse the parameter space in the direction of steepest descent by computing the gradient of the chosen objective function (Section 2) with respect to the parameters. For MMD, automatic differentiation can be employed; whereas for Wasserstein distance, the gradient can be computed using Equation 10. This gradient can then be used to perform optimization using standard algorithms like Adam. However, gradient measurements for quantum circuits scale poorly with the system size.

Another problem with standard gradient descent algorithms is that each optimization step is strongly connected to a Euclidean geometry on the parameter space. The natural gradient is a generalization of a typical gradient that accounts for the curvature of the metric function at hand. The Quantum Natural Gradient (QNG) [6] is the quantum version of the natural gradient that uses the Fubini-Study metric tensor to take into account the geometry of the quantum state space. However, constructing the Fubini-Study metric tensor requires $\mathcal{O}(N^2)$ measurements in qubits, which is infeasible for large systems.

4.1.1 Benchmarking Gradient-based variants

A preliminary benchmark of several gradient-based optimizers was conducted on a 4-qubit system to identify the most robust choice for larger simulations. The optimizers included Adam (baseline), RAdam, NAdam, AdamW, AdaBelief, Lion, and AMSGrad.

The results indicated that **AdaBelief** and **AMSGrad** significantly outperformed the standard Adam optimizer in terms of convergence speed and final fidelity. Further testing on a 6-qubit system confirmed that AMSGrad provided the most stable performance across different initializations. Consequently, AMSGrad was selected as the primary optimizer for the benchmarking of architectures.

These reasons provide motivation for exploring gradient-free optimizers.

4.2 Gradient-free optimizers

These optimizers do not require gradient information and instead rely on evaluating the objective function at different points in the parameter space to guide the search for optimal parameters.

4.2.1 Rotosolve

Rotosolve is a coordinate-wise optimization algorithm that iteratively optimizes one parameter at a time while keeping others fixed [7] using the following update rule for the d -th parameter:

$$\theta_d^* = -\frac{\pi}{2} - \arctan2\left(2\langle H \rangle_{\theta_d=0} - \langle H \rangle_{\theta_d=\pi/2} - \langle H \rangle_{\theta_d=-\pi/2}, \langle H \rangle_{\theta_d=\pi/2} - \langle H \rangle_{\theta_d=-\pi/2}\right) \quad (11)$$

This is targeted towards optimization problems where the encoded objective function can be represented as a Hermitian operator. Hence, it is not applicable for the present use case where the objective function is a distance metric between distributions, and not an expectation value of a Hermitian operator.

4.2.2 Simultaneous Perturbation Stochastic Approximation (SPSA)

SPSA estimates the gradient by perturbing all parameters simultaneously in random directions [8] with the following update rule:

$$\theta_{k+1} = \theta_k - a_k \frac{f(\theta_k + c_k \Delta_k) - f(\theta_k - c_k \Delta_k)}{2c_k} \begin{bmatrix} \Delta_{k1}^{-1} \\ \Delta_{k2}^{-1} \\ \vdots \\ \Delta_{kp}^{-1} \end{bmatrix} \quad (12)$$

Only two evaluations of the objective function per iteration are required, regardless of the number of parameters, making it efficient for high-dimensional optimization problems. However, it can be noisy and may require careful tuning of hyperparameters a_k and c_k .

4.2.3 Quantum Natural SPSA (QNSPSA)

QNSPSA [9] manages to combine the merits of QNG and SPSA by estimating both the gradient and the metric tensor stochastically. The gradient is estimated in the same fashion as the SPSA algorithm, while the Fubini-Study metric is computed by a second-order process. In practice, it requires 2 (for gradient) + 4 (for metric tensor) + 2 (for the current and the next-step loss) = 8 circuit evaluations per iteration, which is still $\mathcal{O}(1)$. Given a randomly sampled direction $\mathbf{h} \sim \mathcal{U}(\{-1, 1\}^d)$, the update rule is as follows:

$$\theta^{(k+1)} = \theta^{(k)} - \eta \widehat{\mathbf{g}}^{-1}(\theta^{(k)}, \mathbf{h}_1^{(k)}, \mathbf{h}_2^{(k)})_{\text{SPSA}} \widehat{\nabla} f(\theta^{(k)}, \mathbf{h}^{(k)})_{\text{SPSA}} \quad (13)$$

where

$$\widehat{\mathbf{g}}(\theta, \mathbf{h}_1, \mathbf{h}_2)_{\text{SPSA}} = \frac{\delta \mathcal{F}}{8\epsilon^2} (\mathbf{h}_1 \mathbf{h}_2^T + \mathbf{h}_2 \mathbf{h}_1^T) \quad (14)$$

$$\delta \mathcal{F} = \mathcal{F}(\theta, \theta + \epsilon \mathbf{h}_1 + \epsilon \mathbf{h}_2) - \mathcal{F}(\theta, \theta + \epsilon \mathbf{h}_1) - \mathcal{F}(\theta, \theta - \epsilon \mathbf{h}_1 + \epsilon \mathbf{h}_2) + \mathcal{F}(\theta, \theta - \epsilon \mathbf{h}_1) \quad (15)$$

$$\mathcal{F}(\theta, \theta') = \mathbb{E}_{|\varphi\rangle \in \mathcal{S}_1, |\psi\rangle \in \mathcal{S}_2} |\langle \varphi(\theta') | \psi(\theta) \rangle|^2 \quad (16)$$

Further details on the algorithm and implementation can be found in [10]. The primary difference between the present implementation and that of [10] is that an average over all states in the ensemble is taken when computing $\mathcal{F}$. This is a simplification that is not ideal and warrants further investigation.

5 Implementation details

The simulation framework for the models discussed in this work was implemented from scratch in the **Julia** programming language [11]. **Julia** was chosen for its high-performance capabilities, particularly for numerical and scientific computing, while maintaining a high-level syntax suitable for rapid prototyping.

5.1 Software Stack

- **Quantum Simulation:** Yao.jl [12] is used, which is a flexible and extensible framework for quantum algorithm research. It provides efficient state vector simulation and allows for the construction of complex, hierarchical quantum circuits.
- **Automatic Differentiation (AD):** To train the Parameterized Quantum Circuits (PQCs), Zygote.jl [13] is employed, which is a source-to-source AD library in **Julia** used for high-level differentiability of the training loop and the IPOT-based Wasserstein distance computation.

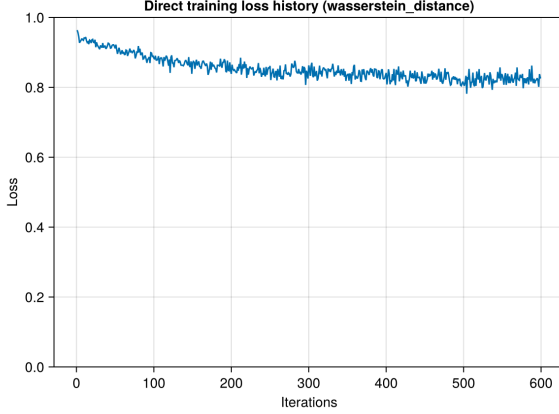
5.2 Computational Optimizations

To address the exponential scaling of quantum state simulation, several optimization strategies were employed:

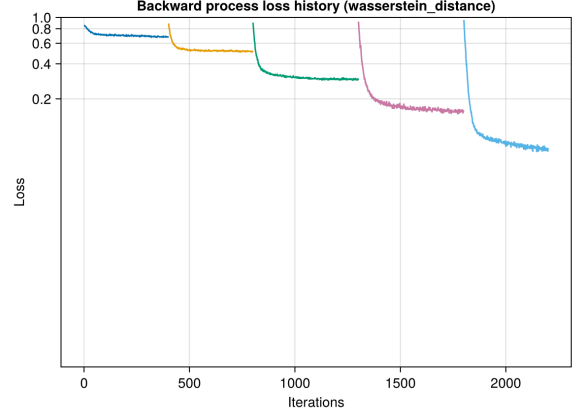
- **IPOT with Envelope Theorem:** As derived in Section 2, the envelope theorem is used to avoid backpropagating through the iterative optimal transport solver. This significantly reduces the computational overhead when using Wasserstein distance as the loss function.
- **In-place Operations:** Wherever possible, in-place operations were used to minimize memory allocations and reduce garbage collection overhead.

6 Benchmarking architectures

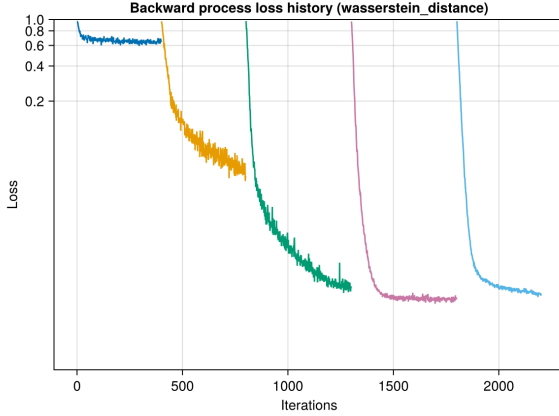
The performance of the three architectures — QuDT, QuDDPM, and S-QuDT — is evaluated on a 5-qubit system. The models are trained to learn 5-qubit clustered states. For all models, $n_a = 3$ ancillary qubits are used. AMSGrad optimizer with a learning rate of $\eta = 0.01$ and QNSPSA are compared for S-QuDT. To ensure a fair comparison, the total circuit depth is kept constant at 80 layers for all models. For QuDDPM and S-QuDT, $T = 5$ steps are used in the reverse process, which means that each step is allocated 16 layers. For QuDT, all 80 layers are used in a single step.



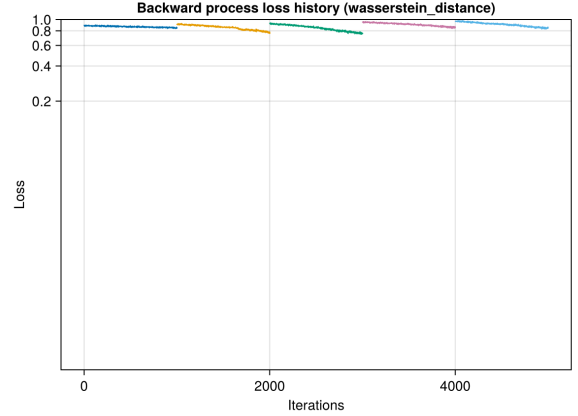
(a) QuDT ($T = 1$, $L = 80$)



(b) QuDDPM ($T = 5$, $L = 16$)
(Loss (y-axis) is on a logarithmic scale)



(c) S-QuDT with AMSGrad ($T = 5$, $L = 16$)
(Loss (y-axis) is on a logarithmic scale)



(d) S-QuDT with QNSPSA ($T = 5$, $L = 16$)
(Loss (y-axis) is on a logarithmic scale)

Figure 6: Comparison of training loss history for different architectures on a 5-qubit system. Figure 6a, Figure 6b, and Figure 6c use AMSGrad whereas Figure 6d uses QNSPSA. S-QuDT with AMSGrad demonstrates the fastest and most stable convergence. The spikes in the loss history of QuDDPM and S-QuDT are due to the sequential training process, where each step is trained after the previous step has been fully trained.

The results are summarized in Table 1.

Model	Optimizer	Total epochs	Final Loss (W_2)
QuDT	AMSGrad	600	0.8245
QuDDPM	AMSGrad	2200	0.0742
S-QuDT	AMSGrad	2200	0.0045
S-QuDT	QNPSA	5000	0.8448

Table 1: Summary of benchmarking results for 5-qubit state generation.

As observed, the single-step QuDT model fails to converge to a low loss, despite having a deep circuit. This suggests that the landscape for a single deep PQC is extremely difficult to navigate. In contrast, both sequential models (QuDDPM and S-QuDT) achieve much higher fidelity. S-QuDT with QNPSA fails to converge but since it is a global optimization method, it may perform better when training all the PQC blocks simultaneously. This needs further investigation. S-QuDT with AMSGrad, in particular, achieves an order of magnitude lower loss than QuDDPM for the same training time and circuit depth, suggesting that the direct transport objective with gradient based optimization is more efficient than denoising when applied sequentially.

7 Future Work: Quantum Kicked Rotor

The Hamiltonian of the QKR is given by

$$i\hbar \frac{\partial}{\partial t} \psi(x, t) = \underbrace{-\frac{\hbar^2}{2I} \frac{\partial^2}{\partial x^2}}_{H_{\text{free}}} \psi(x, t) + \underbrace{k \cos(x) \sum_n \delta(t - nT)}_{H_{\text{kick}}} \psi(x, t) \quad (17)$$

where x is the angular position of the particle, p is the conjugate momentum, k is the kick strength, and T is the period of the kicks. The evolution over one period can be described by the Floquet operator $\hat{U}$ as follows:

$$\begin{aligned} \hat{U} &= \hat{U}_{\text{kick}} \hat{U}_{\text{free}} \\ &= \exp\left(-i \frac{K}{\hbar_s} \cos \hat{x}\right) \exp\left(-i \frac{\hat{p}^2}{2\hbar_s}\right) \end{aligned} \quad (18)$$

where $\hbar_s = \frac{\hbar T}{I}$, $K = \frac{k}{\hbar}$. Detailed derivation of the Floquet operator can be found in [Appendix B](#).

Long time dynamics are generated by successive applications of the Floquet operator. The QKR exhibits a phenomenon called dynamical localization, where the state becomes exponentially localized in momentum space after a certain time, despite the classical counterpart exhibiting unbounded energy growth. This makes it an interesting testbed for generative models to learn and sample from.

A primary objective now is to scale the generative process to 10-qubit systems (with 5 ancilla qubits), since that is the minimum system size required for the QKR to exhibit its characteristic behavior. In this regime, the Hilbert space dimension grows to $2^{10+5} = 32,768$ and correspondingly, the matrices grow to *billions* of elements. To handle this would require significant computational resources and optimizations, which are currently being worked on.

Bibliography

- [1] B. Zhang, P. Xu, X. Chen, and Q. Zhuang, “Generative Quantum Machine Learning via Denoising Diffusion Probabilistic Models,” *Phys. Rev. Lett.*, vol. 132, no. 10, p. 100602, Mar. 2024, [Online]. Available: <http://dx.doi.org/10.1103/PhysRevLett.132.100602>
- [2] M. Cuturi, “Sinkhorn Distances: Lightspeed Computation of Optimal Transportation Distances,” 2013, [Online]. Available: <https://arxiv.org/abs/1306.0895>
- [3] Y. Xie, X. Wang, R. Wang, and H. Zha, “A Fast Proximal Point Method for Computing Exact Wasserstein Distance.” [Online]. Available: <https://arxiv.org/abs/1802.04307>
- [4] R. Belyansky, P. Bienias, Y. A. Kharkov, A. V. Gorshkov, and B. Swingle, “Minimal Model for Fast Scrambling,” *Physical Review Letters*, vol. 125, no. 13, 2020, doi: [10.1103/physrevlett.125.130601](https://doi.org/10.1103/physrevlett.125.130601).
- [5] A. Kandala *et al.*, “Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets,” *Nature*, vol. 549, no. 7671, pp. 242–246, 2017, doi: [10.1038/nature23879](https://doi.org/10.1038/nature23879).
- [6] J. Stokes, J. Izaac, N. Killoran, and G. Carleo, “Quantum Natural Gradient,” *Quantum*, vol. 4, p. 269, May 2020, [Online]. Available: <http://dx.doi.org/10.22331/q-2020-05-25-269>
- [7] M. Ostaszewski, E. Grant, and M. Benedetti, “Structure optimization for parameterized quantum circuits,” *Quantum*, vol. 5, p. 391, Jan. 2021, doi: [10.22331/q-2021-01-28-391](https://doi.org/10.22331/q-2021-01-28-391).
- [8] J. Spall, “Multivariate stochastic approximation using a simultaneous perturbation gradient approximation,” *IEEE Transactions on Automatic Control*, vol. 37, no. 3, pp. 332–341, 1992, doi: [10.1109/9.119632](https://doi.org/10.1109/9.119632).
- [9] J. Gacon, C. Zoufal, G. Carleo, and S. Woerner, “Simultaneous Perturbation Stochastic Approximation of the Quantum Fisher Information,” *Quantum*, vol. 5, p. 567, Oct. 2021, doi: [10.22331/q-2021-10-20-567](https://doi.org/10.22331/q-2021-10-20-567).
- [10] Y. Duan, “Quantum natural SPSA optimizer.” Xanadu, 2022.
- [11] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, “**Julia**: A fresh approach to numerical computing,” *SIAM Review*, vol. 59, no. 1, pp. 65–98, Sept. 2017, doi: [10.1137/141000671](https://doi.org/10.1137/141000671).
- [12] X.-Z. Luo, J.-G. Liu, Z. Pan, and L. Wang, “Yao.jl: Extensible, Efficient Framework for Quantum Algorithm Design,” *Quantum*, vol. 4, Oct. 2020, doi: [10.22331/q-2020-10-11-341](https://doi.org/10.22331/q-2020-10-11-341).
- [13] M. Innes, “Don't Unroll Adjoint: Differentiating SSA-Form Programs,” *CoRR*, 2018, [Online]. Available: <http://arxiv.org/abs/1810.07951>
- [14] S. Prince, *Understanding deep learning*. London, England: MIT Press, 2023.

A Classical DDPM

The idea behind Denoising Diffusion Probabilistic Models (or Diffusion Models for short) is to destroy the training data by adding noise, and then learn to reverse this process to generate new data.

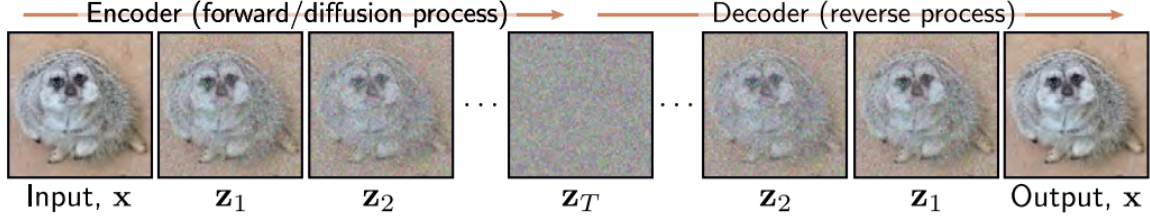


Figure 7: From the book Understand Deep Learning [14]

Let us consider the case of image generation. There exist many manifolds in the high-dimensional space of images, that contains all valid images. The aim is to sample new images from these manifolds, but the problem is that their algebraic form is not known because it is too complex. Lets consider any such manifold; take a point on it and add noise to it iteratively, pushing it outside the manifold. Then a model is trained to learn to remove the noise step by step, to get back into the manifold. This is the basic idea behind diffusion models. The process of adding noise is called the *forward process*, and the process of removing noise is called the *reverse process*.

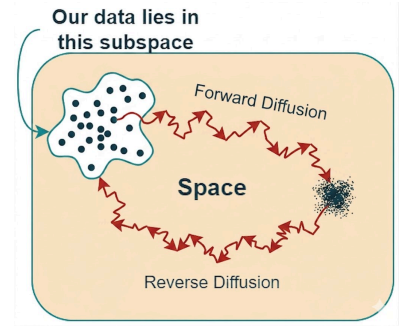


Figure 8: A high level conceptual overview of DDPM

B Derivation of the Floquet operator

Starting from the time-dependent Schrodinger equation given in Equation 17, let the Floquet operator $\hat{U}$ advance the system from $t = nT^+$ to $t = (n + 1)T^+$, where n is an integer. T^+ represents the time just after the kick whereas T^- represents the time just before the kick. The evolution can be broken down into two steps:

1. $t = nT^+$ to $t = (n + 1)T^-$

During this time interval, the system evolves under the free particle Hamiltonian, which gives the evolution operator

$$\begin{aligned}\hat{U}_{\text{free}} &= \exp\left(-\frac{i}{\hbar} \int_{nT^+}^{(n+1)T^-} \hat{H}_{\text{free}} dt\right) \\ &= \exp\left(-i \frac{T}{\hbar} \frac{\hat{p}^2}{2I}\right)\end{aligned}\tag{19}$$

2. $t = (n + 1)T^-$ to $t = (n + 1)T^+$

During this time interval, the system evolves under the kick Hamiltonian. The kick is instantaneous, so the evolution operator can be computed as:

$$\begin{aligned}
\hat{U}_{\text{kick}} &= \exp\left(-\frac{i}{\hbar} \int_{(n+1)T^-}^{(n+1)T^+} \hat{H}_{\text{kick}} dt\right) \\
&= \exp\left(-i \frac{k}{\hbar} \cos \hat{x}\right)
\end{aligned} \tag{20}$$

Combining these two steps, the Floquet operator is given by

$$\begin{aligned}
\hat{U} &= \hat{U}_{\text{kick}} \hat{U}_{\text{free}} \\
&= \exp\left(-i \frac{k}{\hbar} \cos \hat{x}\right) \exp\left(-i \frac{T}{\hbar} \frac{\hat{p}^2}{2I}\right)
\end{aligned} \tag{21}$$

Substituting $\hbar_s = \frac{\hbar T}{I}$ and $K = \frac{k}{\hbar}$, the Floquet operator can be rewritten in the form given in [Equation 18](#).